



AIQUAA · AUTOMATIZACIÓN DE PRUEBAS DE SOFTWARE

Patrón Pre-request + Post-request con validación SQL en Postman

Guía paso a paso para entender y reutilizar el patrón de automatización end-to-end de la carpeta `E2E - Flujos con validación SQL`: cómo se declara una variable/función global una sola vez, cómo se arma el body de un request a partir de una plantilla, y cómo se valida antes y después contra la base de datos.

Profesor: Steven Gilbert Gracia Ayala

Curso: Automatización de Pruebas de Software

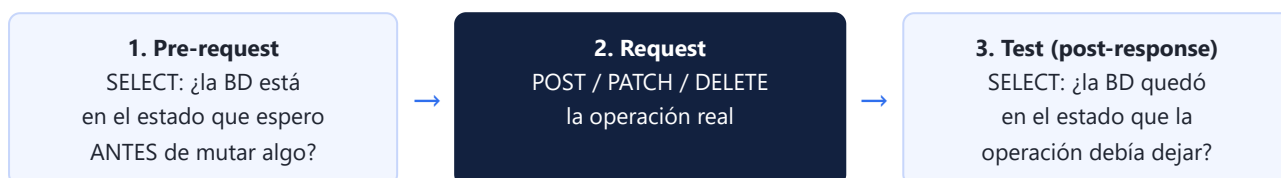
Colección: `postman_collection.json` · API: aiqaaa Sandbox API

Basado en el patrón usado en colecciones reales de automatización bancaria (SPI/CORE).

1 ¿Qué problema resuelve este patrón?

Un test de API "normal" solo revisa la respuesta HTTP: código de estado y forma del JSON. Eso no prueba que la operación realmente haya hecho lo que dice hacer en la base de datos. Un endpoint podría devolver **201 Created** con un JSON perfecto y, por un bug, no haber insertado nada — o haber insertado el dato con el estado equivocado.

El patrón agrega una segunda verificación, **directa contra la base**, en dos momentos:



Por qué importa: si el paso 3 solo mirara la respuesta del paso 2, un bug que devuelve un 201 falso positivo (sin insertar nada) pasaría desapercibido. El SELECT final es la prueba real.

2 Vocabulario de Postman que usa el patrón

Concepto	Qué es	Dónde se edita en Postman
Pre-request Script	Código JS que corre <i>antes</i> de mandar el request.	Puede estar en la colección (corre antes de TODOS los requests) o en un request individual (pestaña "Pre-request Script" de ese item).
Tests Script	Código JS que corre <i>después</i> de recibir la respuesta.	Pestaña "Tests" (o "Scripts > Post-response") de cada request.
Collection Variable	Variable compartida por toda la colección, visible desde cualquier request/script.	<code>pm.collectionVariables.get/set(...)</code>
pm.variables	Variable "local", vive solo durante la ejecución de un mismo request (su pre-request y su test comparten este scope).	<code>pm.variables.get/set(...)</code>
pm.sendRequest	Manda un HTTP request adicional desde dentro de un script, sin que aparezca como un item más de la colección.	Se usa dentro de pre-request/test para consultar la base vía el endpoint SQL.

3 Paso a paso: qué declara cada componente

3.1 — Pre-request Script de la COLECCIÓN (se declara una sola vez)

Se edita haciendo clic en el nombre de la colección (no en un request) → pestaña **Scripts** → **Pre-request script**. Corre antes de *cada* request de la colección, así que es el lugar correcto para declarar cosas que

todos van a reutilizar:

```
if (typeof utils === "undefined") {
  utils = {
    bodySqlRest: function (sql, params) {
      return {
        url: pm.collectionVariables.get("baseUrl") + "/api/v1/sql/select",
        method: "POST",
        header: {
          "Content-Type": "application/json",
          "x-api-key": pm.collectionVariables.get("apiKey")
        },
        body: { mode: "raw", raw: JSON.stringify({ sql: sql, params: params || [] }) }
      };
    }
  };
}

// Template base del body de "Crear transferencia". Se recalcula en cada
// corrida (no se edita a mano en el JSON de la colección).
pm.collectionVariables.set("transferenciaTemplate", JSON.stringify({
  cuentaOrigenId: 1, cuentaDestinoId: 2, monto: 25000,
  descripcion: "Pago de alquiler - E2E"
})));
```

- A** `utils.bodySqlRest(sql, params)` es una *función*, no una variable de Postman: vive en memoria durante toda la corrida de la colección. Su trabajo es armar el objeto que después se le pasa a `pm.sendRequest(...)` para pegarle a `/api/v1/sql/select` — la misma URL, el mismo header `x-api-key` y el mismo formato de body (`{ sql, params }`) en un solo lugar. Sin esto, cada pre-request/test tendría que repetir esas 10 líneas.
- B** `transferenciaTemplate` sí es una *collection variable* (persiste y es visible desde cualquier request). Es el body "por defecto" de una transferencia válida. Se guarda como texto JSON (`JSON.stringify(...)`) porque las variables de Postman siempre son strings.

Ojo: si esta variable no está declarada — por ejemplo, corriste un solo request suelto en vez de la colección completa desde el Runner — `utils` no va a existir y el script va a tirar `ReferenceError: utils is not defined`. Por eso conviene correr la carpeta completa, no un request aislado, la primera vez.

3.2 — Pre-request Script del REQUEST (uno por item, distinto del anterior)

Esto es la pestaña "Pre-request Script" de ese request puntual. Acá se usa lo que la colección ya declaró:

```
// Clona el template SIN mutarlo (caso feliz) y lo deja listo en
// transferenciaBody, que el Body del request va a leer como {{transferenciaBody}}.
const template = JSON.parse(pm.collectionVariables.get("transferenciaTemplate"));
pm.collectionVariables.set("transferenciaBody", JSON.stringify(template));

// Valida la precondition: que las cuentas existan y esten activas
// ANTES de mandar el POST.
const request = utils.bodySqlRest(
  "SELECT id, activa FROM cuentas WHERE id IN ($1, $2)",
  [template.cuentaOrigenId, template.cuentaDestinoId]
);
```

```
pm.sendRequest(request, (err, res) => {
  const rows = res.json().data || [];
  // ... console.log de diagnostico
});
```

- A** `JSON.parse(pm.collectionVariables.get(...))` — convierte el string guardado de vuelta a objeto JS para poder tocarlo. Esto es clonar el template.
- B** Para un caso **negativo** (ver sección 5), acá se agrega UNA línea que muta el objeto — por ejemplo `template.monto = -500;` — antes de guardarlo. Es el mismo template para todos los casos, solo cambia lo que se muta.
- C** `pm.collectionVariables.set("transferenciaBody", ...)` — guarda la versión final (mutada o no) en la variable que el request va a usar como body.

3.3 — El Body del request: `{{transferenciaBody}}`

Clave del patrón: el campo Body del request no tiene JSON hardcodeado. Tiene literalmente el texto `{{transferenciaBody}}`. Postman reemplaza esa variable por su valor actual justo antes de mandar el request — y ese valor lo acaba de escribir el Pre-request Script un instante antes. Por eso el orden importa: Postman siempre corre el Pre-request Script completo antes de armar/enviar el request.

3.4 — Tests Script (post-response): la segunda validación

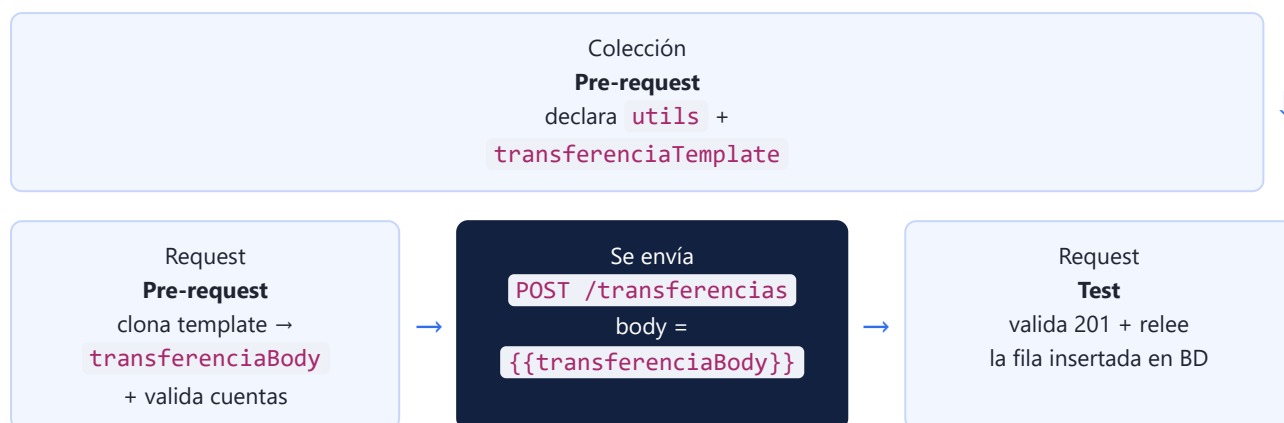
```
pm.test("Status 201 Created", () => { pm.response.to.have.status(201); });

const body = pm.response.json();
const transferenciaId = body.data.id;

// Post-request: vuelve a consultar la BD (mismo helper utils.bodySqlRest)
// para confirmar que el INSERT realmente quedo persistido.
const request = utils.bodySqlRest(
  "SELECT id, cuenta_origen_id, cuenta_destino_id, estado FROM transferencias WHERE id = $1",
  [transferenciaId]
);
pm.sendRequest(request, (err, res) => {
  pm.test("La BD confirma la fila insertada con estado 'pendiente'", () => {
    const row = res.json().data[0];
    pm.expect(row).to.not.be.undefined;
    pm.expect(row.estado).to.eql("pendiente");
  });
});
```

- A** Primer `pm.test`: valida la respuesta HTTP (lo que cualquier test "normal" haría).
- B** Se extrae el `id` que la API devolvió — es el dato que necesitamos para ir a buscar *esa* fila específica en la base.
- C** Segundo `pm.sendRequest` + `pm.test` anidado: este es el que realmente prueba el efecto en la BD. Nota que `pm.test` puede llamarse dentro del callback de `pm.sendRequest` — Postman lo sigue contando como un test más en los resultados, aunque sea asíncrono.

4 Diagrama completo del flujo (caso feliz)



5 Reusar el patrón para un caso negativo

Con el template ya armado, generar un caso de borde es cambiar **una sola línea** en el pre-request y la aserción del test:

Caso	Mutación en el pre-request	Qué se espera	Cómo se prueba en la BD
Monto inválido	<code>template.monto = -500;</code>	Zod (<code>monto: positive()</code>) rechaza con 400 antes de tocar la BD	<code>COUNT(*)</code> de <code>transferencias</code> antes vs. después: debe ser igual (no se insertó nada)
Cuenta origen inexistente	<code>template.cuentaOrigenId = 999999;</code>	Pasa Zod, rompe la <i>foreign key</i> al insertar (Postgres <code>23503</code>) → 400	Mismo <code>COUNT(*)</code> antes/después

```
// Pre-request del caso "monto invalido"
const template = JSON.parse(pm.collectionVariables.get("transferenciaTemplate"));
template.monto = -500; // ← unica linea distinta
pm.collectionVariables.set("transferenciaBody", JSON.stringify(template));

const request = utils.bodySqlRest("SELECT COUNT(*) AS total FROM transferencias");
pm.sendRequest(request, (err, res) => {
  pm.variables.set("totalAntes", res.json().data[0].total); // pm.variables: solo
}); // para este request

// Test del mismo request
pm.test("Status 400 Bad Request (monto invalido)", () => {
  pm.response.to.have.status(400);
});

const request2 = utils.bodySqlRest("SELECT COUNT(*) AS total FROM transferencias");
pm.sendRequest(request2, (err, res) => {
  pm.test("La BD no registro ninguna fila nueva", () => {
    pm.expect(Number(res.json().data[0].total)).to.eql(Number(pm.variables.get("totalAntes")));
  });
});
```

Nota didáctica: acá se usa `pm.variables` (no `pm.collectionVariables`) para `totalAntes`, porque ese dato solo hace falta *dentro de este mismo request* — no necesita viajar en el body ni ser visto por otros items. Guardar en collection variables solo lo que realmente necesita ser global evita que la colección se llene de variables "basura".

6 Los 5 requests de la carpeta E2E - Flujos con validación SQL

Request	Operación	Pre-request valida	Test (post-response) valida
Crear transferencia	INSERT	Cuentas 1 y 2 activas	201 + fila en <code>transferencias</code> con <code>estado='pendiente'</code>
Crear transferencia – monto inválido	INSERT rechazado	Cuenta el total de filas antes	400 + el total no cambió
Crear transferencia – cuenta origen inexistente	INSERT rechazado (FK)	Cuenta el total de filas antes	400 + el total no cambió
Pagar factura	UPDATE transaccional	Factura 3 aún no está <code>'pagada'</code>	200 + <code>facturas.estado='pagada'</code> y fila nueva en <code>pagos</code>
Revocar rol de usuario	Soft-delete (UPDATE)	El rol está activo para ese usuario	204 + <code>usuario_rols.activo=false</code> (la fila sigue existiendo, no se borra)

7 Cómo aplicar el patrón a otro endpoint

- 1 Identificá qué tabla(s) toca el endpoint (ver la tabla de `qa_training` en el `README.md`) y qué columna refleja el resultado de la operación (un `estado`, un `activo`, etc.).
- 2 En el Pre-request Script del request, escribí un `SELECT` con `utils.bodySqlRest(...)` que confirme la precondition (¿existe el dato? ¿está en el estado esperado antes de mutarlo?).
- 3 Si el endpoint tiene un body con varios campos y querés probar casos de borde, armá un `xxxTemplate` en el Pre-request Script de la **colección** (una vez) y clonalo/mutalo en cada item, igual que en la sección 5.
- 4 En el Tests Script, primero afirmá el status code y la forma de la respuesta; después, con el id/clave que devolvió la API, hacé un segundo `utils.bodySqlRest(...)` que confirme el estado final en la tabla.
- 5 Para casos negativos, en vez de leer una fila puntual (todavía no existe), comparar un `COUNT(*)` antes/después es la forma más simple de probar "no pasó nada en la BD".

8 Glosario rápido de `pm.*` usadas

Método	Para qué
--------	----------

<code>pm.collectionVariables.get/set(key, value)</code>	Leer/escribir una variable de colección (persiste entre requests).
<code>pm.variables.get/set(key, value)</code>	Leer/escribir una variable local (solo vive durante ese request).
<code>pm.sendRequest(options, callback)</code>	Manda un HTTP request extra desde un script; el callback recibe <code>(err, response)</code> .
<code>pm.response.json()</code>	Parsea el body de la respuesta como JSON.
<code>pm.response.to.have.status(n)</code>	Asercion de status code.
<code>pm.test(nombre, fn)</code>	Declara un test con nombre; aparece en los resultados del Runner/Newman.
<code>pm.expect(valor).to...</code>	Aserciones estilo Chai (<code>.eq</code> , <code>.to.not.be.undefined</code> , etc.).

Este documento acompaña la carpeta `E2E - Flujos con validación SQL` de `postman_collection.json` en el repositorio del curso. Cualquier duda sobre una línea puntual de los scripts, comparala contra el request real en Postman — los comentarios `//` dentro de cada script siguen la misma numeración de pasos que esta guía.